

Stacks, Queues, and Linked Lists

Dynamic Sets: $\rightarrow$ sets that change over time
 $\rightarrow$ Operations: search, insert, remove, etc.

Stacks: — A dynamic set that implements LIFO scheme

Last-in, first out

- Insert operation is called PUSH
- Delete operation is called POP

Array based implementation

$\rightarrow$ A stack of n elements with an array $S[1..n]$

$\rightarrow$ Attributes: $S.top \rightarrow$ index of most recently pushed element

$\rightarrow S[1]$ holds the bottom of the stack

$\rightarrow S.top = 0$, then the stack is empty

$\rightarrow$ "Underflow" when popping from empty stack

$\rightarrow$ "overflow" when $S.top$ exceeds n

Stack - Empty (s)

if $S.top == 0$
 return true
else return false

$O(1)$

Push (S, x)

$S.top \leftarrow S.top + 1$
 $S[S.top] \leftarrow x$

$O(1)$

check
for
overflow!

Pop (s)

if Stack - Empty (s)
 return "Underflow"

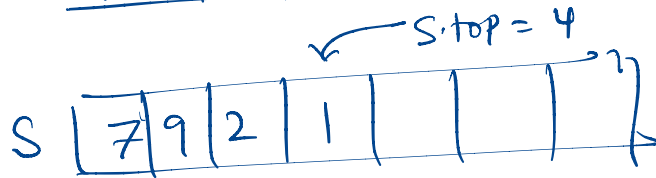
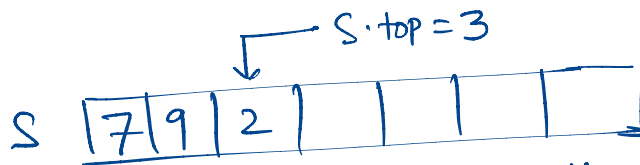
else

$S.top \leftarrow S.top - 1$

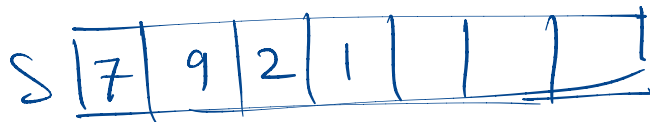
return $S[S.top + 1]$

$O(1)$

→ Example :



Push (S, 1)



Pop (S)

$S.top = 3$

Queues : A dynamic set that implements FIFO policy

first-in, first out

→ Insert op. is called Enqueue → Happens at the tail

→ Delete op. is called Dequeue → Happens at the head

An array based implementation:

→ A queue of at most $n-1$ elements using $Q[1...n]$

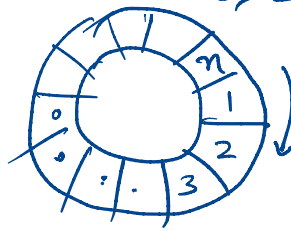
→ Attributes: $Q.head$ ← location of dequeue

$Q.tail$ ← next location of enqueue

→ Elements in Q are between $Q.head$ and $Q.tail-1$

→ "wrap around" the array in a circular fashion

⇒ so, 1 follows n



≈

1	2	3	.	.	.	.	.	.	n
---	---	---	---	---	---	---	---	---	---

Use to check for "underflow"

→ Queue is empty when $Q.head == Q.tail$

→ Queue is full when $Q.head == Q.tail + 1$

Use to check for "overflow"

→ Initially, $Q.head = Q.tail = 1$

Assume $n = Q.length$

Enqueue (Q, x)

$Q[Q.tail] \leftarrow x$

if $Q.tail == Q.length$

$Q.tail \leftarrow 1$ // wrap around & update

else $Q.tail \leftarrow Q.tail + 1$ // update

Dequeue (Q)

$x \leftarrow Q[Q.head]$

if $Q.head == Q.length$

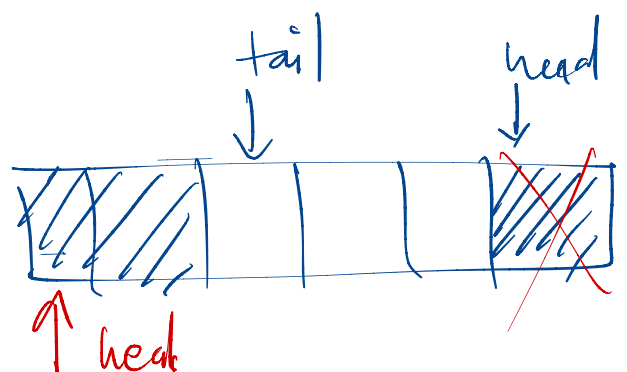
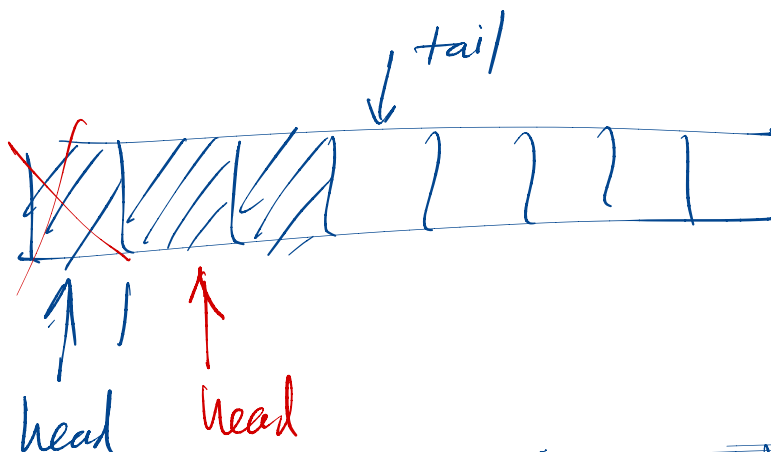
$Q.head \leftarrow 1$ // wrap around and increment

else

$Q.head \leftarrow Q.head + 1$

return x

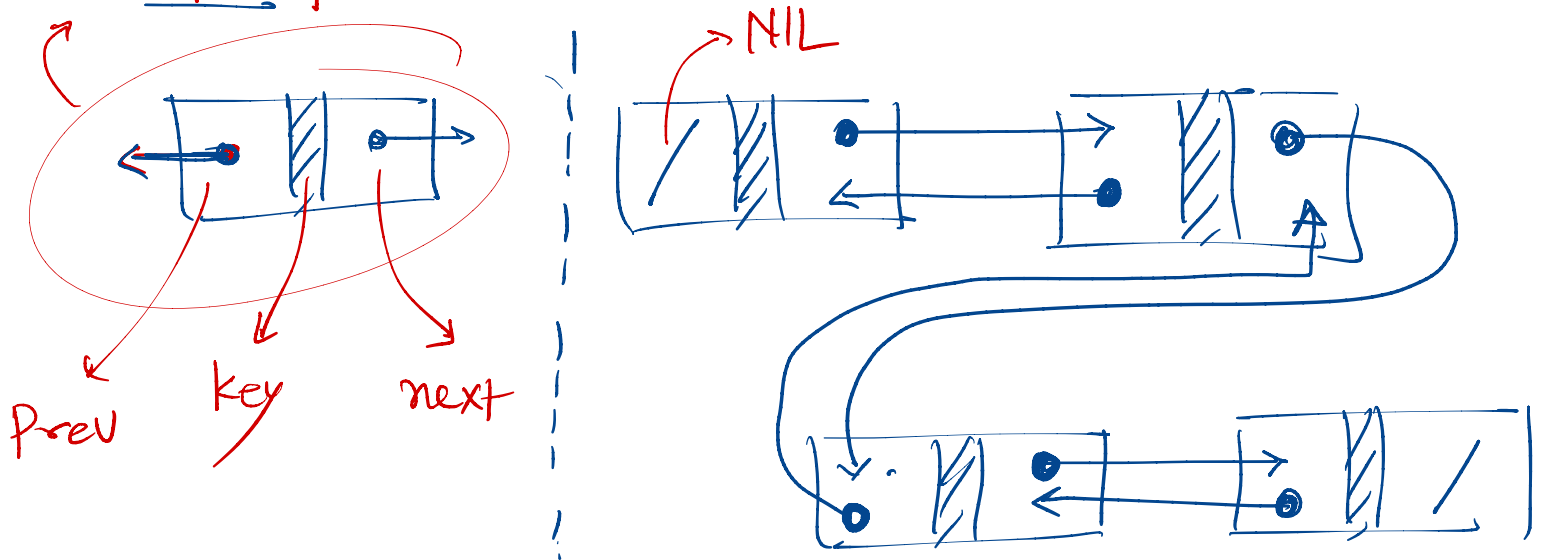
Check for
underflow
&
overflow



Linked List

- A data structure where objects are arranged in a linear order
- The order is determined by pointers

an object of the linked list



Attributes of an element x :

$x.next$: Points to successor

$x.prev$: Points to predecessor

key : data

numbers
some
satellite
data
(it's a pointer)

- if $\overbrace{x.prev}^{\text{Head}} == \text{NIL} \Rightarrow$ No predecessor

- if $\underbrace{x.next}_{\text{tail}} == \text{NIL} \Rightarrow$ No successor

Denoting the linked list by L

$L.head \rightarrow$ points to first object

$L.tail \rightarrow$ points to last object

Searching a linked list

$\rightarrow$ find the first instance of a key k in L

$\rightarrow$ return NIL if not found.

List-Search(L, k)

$x \leftarrow L.head$

While $x \neq NIL$ and $x.key \neq k$

$x \leftarrow x.next$

return x $\rightarrow$ an object of L

containing key k

$\rightarrow \theta(n)$

Inserting into the linked list

→ Splice element x (whose key has been set) onto the front of L (head)

List-Insert(L, x)

$x \cdot \text{next} \leftarrow L \cdot \text{head}$

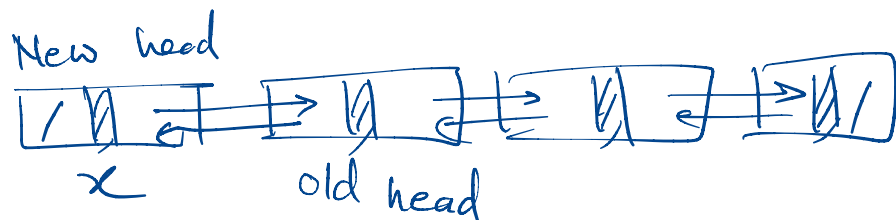
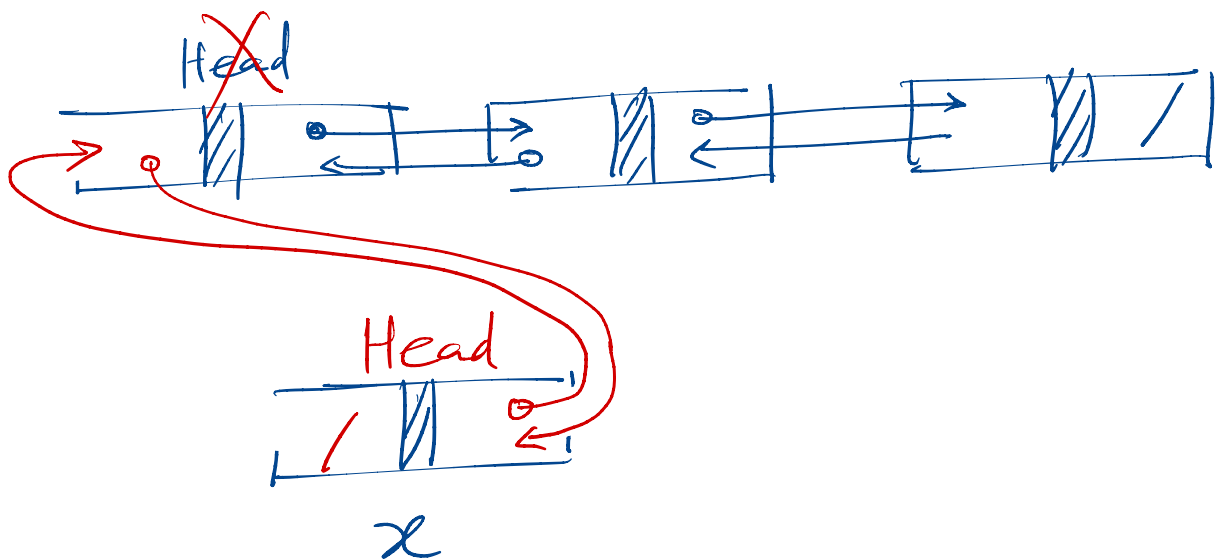
if $L \cdot \text{head} \neq \text{NIL}$

$L \cdot \text{head} \cdot \text{prev} = x$

$L \cdot \text{head} \leftarrow x$

$x \cdot \text{prev} \leftarrow \text{NIL}$

$\rightarrow O(1)$



Deleting an element from L

→ removes an element x from L

→ first call List-Search to identify x in L for a given key.

List-Delete (L, x)

if $x \cdot \text{prev} \neq \text{NIL}$

$x \cdot \text{prev} \cdot \text{next} \leftarrow x \cdot \text{next}$

else $L \cdot \text{head} \leftarrow x \cdot \text{next}$

if $x \cdot \text{next} \neq \text{NIL}$

$x \cdot \text{next} \cdot \text{prev} \leftarrow x \cdot \text{prev}$

